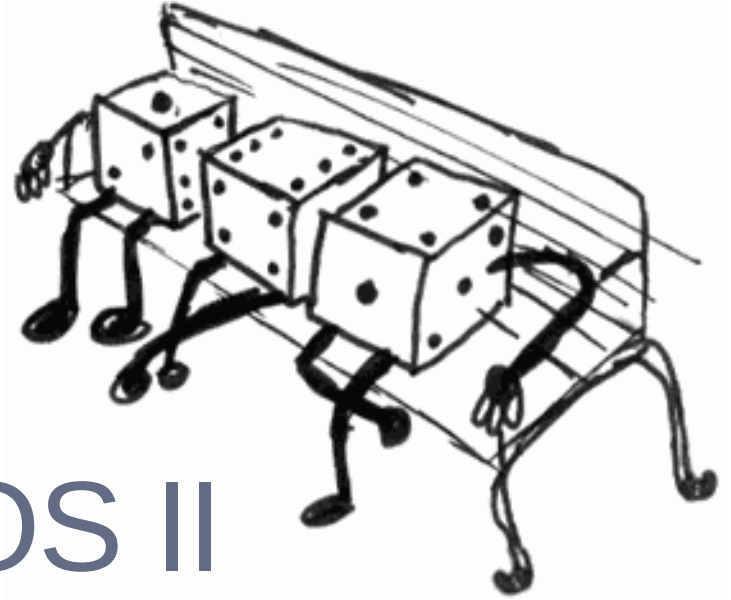




BANCO DE DADOS II

Prof. Michel Junio Ferreira Rosa
Ciência da Computação



VIEWS (VISÕES)

Views

- Considerações sobre segurança podem exigir que determinados dados não estejam disponíveis para alguns usuários;
- Pode-se criar uma coleção de **relações personalizadas** que se ajustem melhor às necessidades dos usuários e que levem em conta a questão de segurança;

Views (Visões)

- Tais relações são chamadas de **VIEWS (VISÕES)**



Views (Visões)

- Uma visão, na terminologia SQL, é uma tabela que é derivada de outras tabelas (tabelas bases ou outras visões previamente definidas);
- Uma visão não está, necessariamente, fisicamente armazenada no Banco de Dados;

Resumindo - Views

- Podem ser vistas como *queries* (consultas) armazenadas e, uma vez criadas, são tratadas como tabelas;



Sintaxe de criação

CREATE [OR REPLACE] VIEW

<nome_da_view> [<lista_de_colunas>]

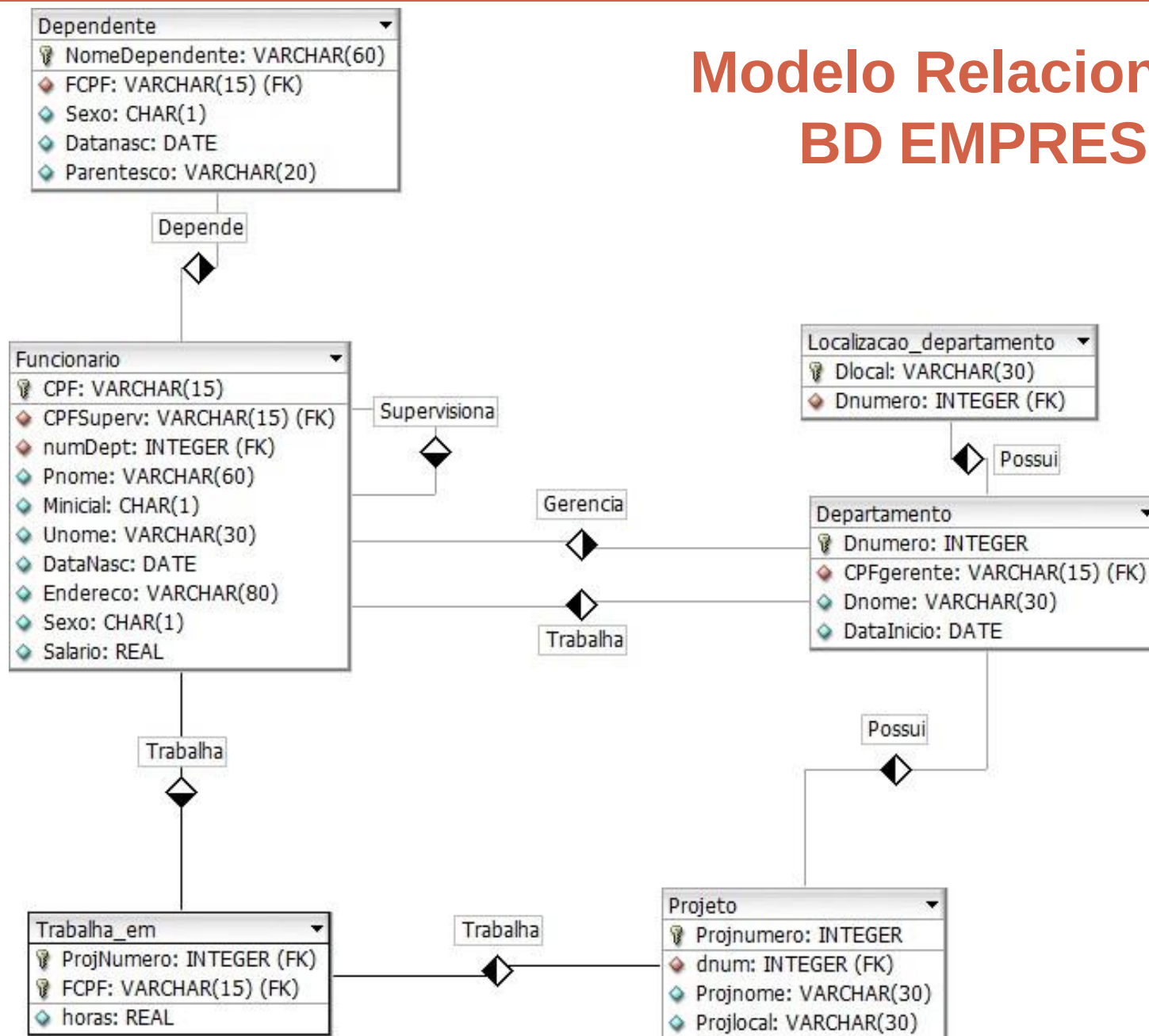
AS <expressão_de_consulta>;

Exemplo

- Criar uma visão que mostre o primeiro nome e o endereço de todos os funcionários que trabalham no projeto de nome “Produto X”.
- Vamos ao modelo...



Modelo Relacional do BD EMPRESA



Consulta

```
SELECT f.pnome, f.endereco  
FROM funcionario as f JOIN  
trabalha_em as t ON f.cpf = t.fcpf  
JOIN projeto as p ON t.projnumero =  
p.projnumero  
WHERE p.projnome = "Produto X";
```

Exemplo

```
CREATE VIEW    empregados_produtoX  
(nome, endereco) AS  
SELECT f.pnome, f.endereco  
FROM funcionario as f JOIN trabalha_em  
as t ON f.cpf = t.fcpf JOIN projeto as  
p ON t.projnumero = p.projnumero  
WHERE p.projnome = "Produto X"
```

Exemplo

```
CREATE VIEW empregados_produtoX  
(nome, endereco) AS  
SELECT f.pnome, f.endereco  
FROM funcionario as f JOIN  
trabalha_em as t ON f.cpf = t.fcpf  
JOIN projeto as p ON t.projnumero =  
p.projnumero  
WHERE p.projnome = "Produto X"
```

Exemplo – Executar uma VIEW

```
SELECT * FROM empregados_produtoX;
```



Exemplo – Alterar uma VIEW

CREATE OR REPLACE VIEW

empregados_projetoX (nome, endereco,
salario) **AS**

SELECT f.pnome, f.endereco, f.salario

FROM funcionario as f **JOIN**

trabalha_em as t **ON** f.cpf = t.fcpf

JOIN projeto as p **ON** t.projnumero =
p.projnumero

WHERE p.projnome = "Produto X"

Exemplo – Remover uma VIEW

- DROP VIEW empregados_produtoX;



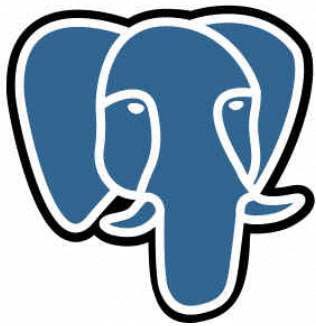
Na prática (VIEWS X SGBDs)

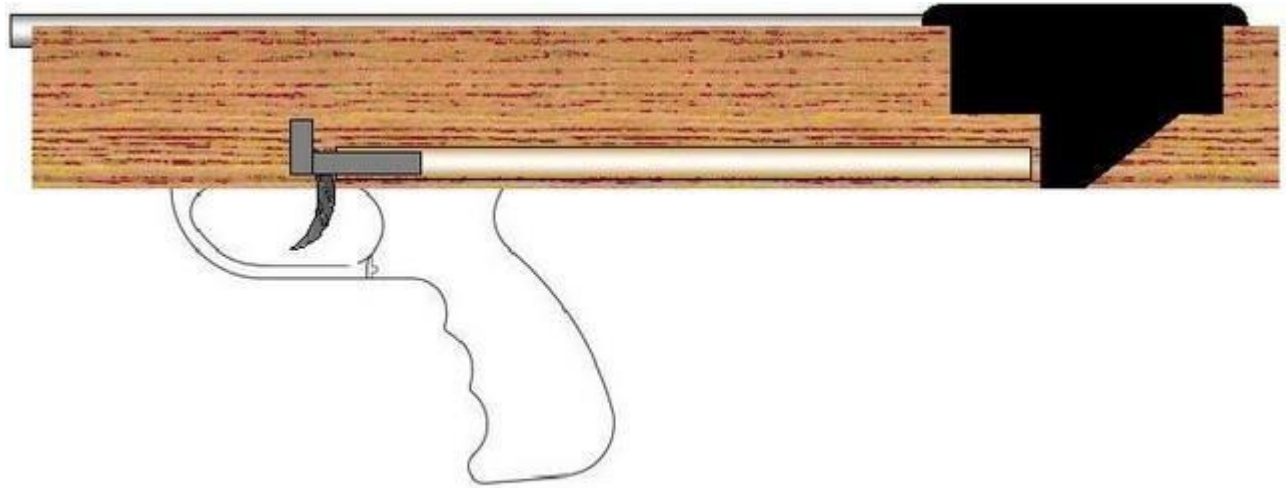
- O comando para criar VIEW é o mesmo para todos os SGBDs (MySQL, PostgreSQL, SQL Server e Oracle)

ORACLE®

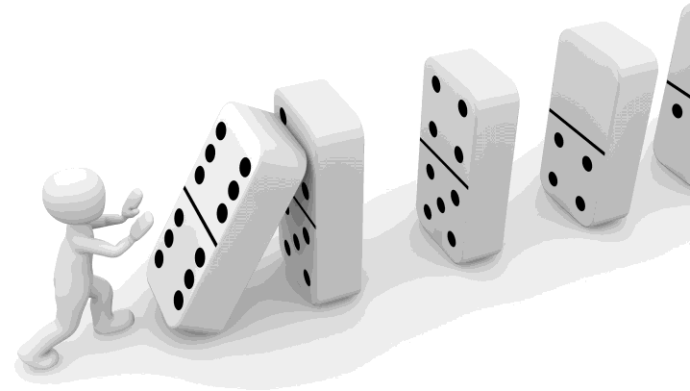
Microsoft®
SQL Server® 2008

PostgreSQL





TRIGGER (GATILHO)



Triggers (gatilhos)

- As *Triggers* (gatilhos) são recursos dos bancos de dados que, executam funções no banco, quando um determinado evento ocorrer na tabela em que a *trigger* estiver associada;
- Os gatilhos podem ser acionados antes (*BEFORE*) ou depois (*AFTER*) de alguma ação de *INSERT*, *UPDATE* e *DELETE*;

Triggers (gatilhos)

- Os triggers são usados para realizar tarefas relacionadas com:
 - Validações;
 - Restrições de acesso;
 - Rotinas de segurança; e
 - Consistência de dados
- Desta forma estes controles deixam de ser executados pela aplicação e passam a ser executados pelos Triggers.

Triggers (gatilhos)

- A criação de um Trigger envolve duas etapas:
 - Um comando SQL que vai disparar o Trigger (INSERT, DELETE, UPDATE);
 - A ação que o Trigger vai executar (Geralmente um bloco de códigos SQL ou Procedimentos Armazenados)

Sintaxe (SQL) – Criar

```
CREATE TRIGGER <nome_da_trigger>  
ON <nome_tabela>  
{ FOR | AFTER | BEFORE | INSTEAD OF }  
{ [INSERT] [DELETE] [UPDATE] }  
AS <comandos_sql>
```

Sintaxe(SQL) –Alterar

```
ALTER TRIGGER <nome> ON <tabela>  
RENAME TO <novo_nome>
```

Sintaxe(SQL) – Excluir

```
DROP TRIGGER <nome> ON <tabela>  
[CASCADE | RESTRICT ]
```

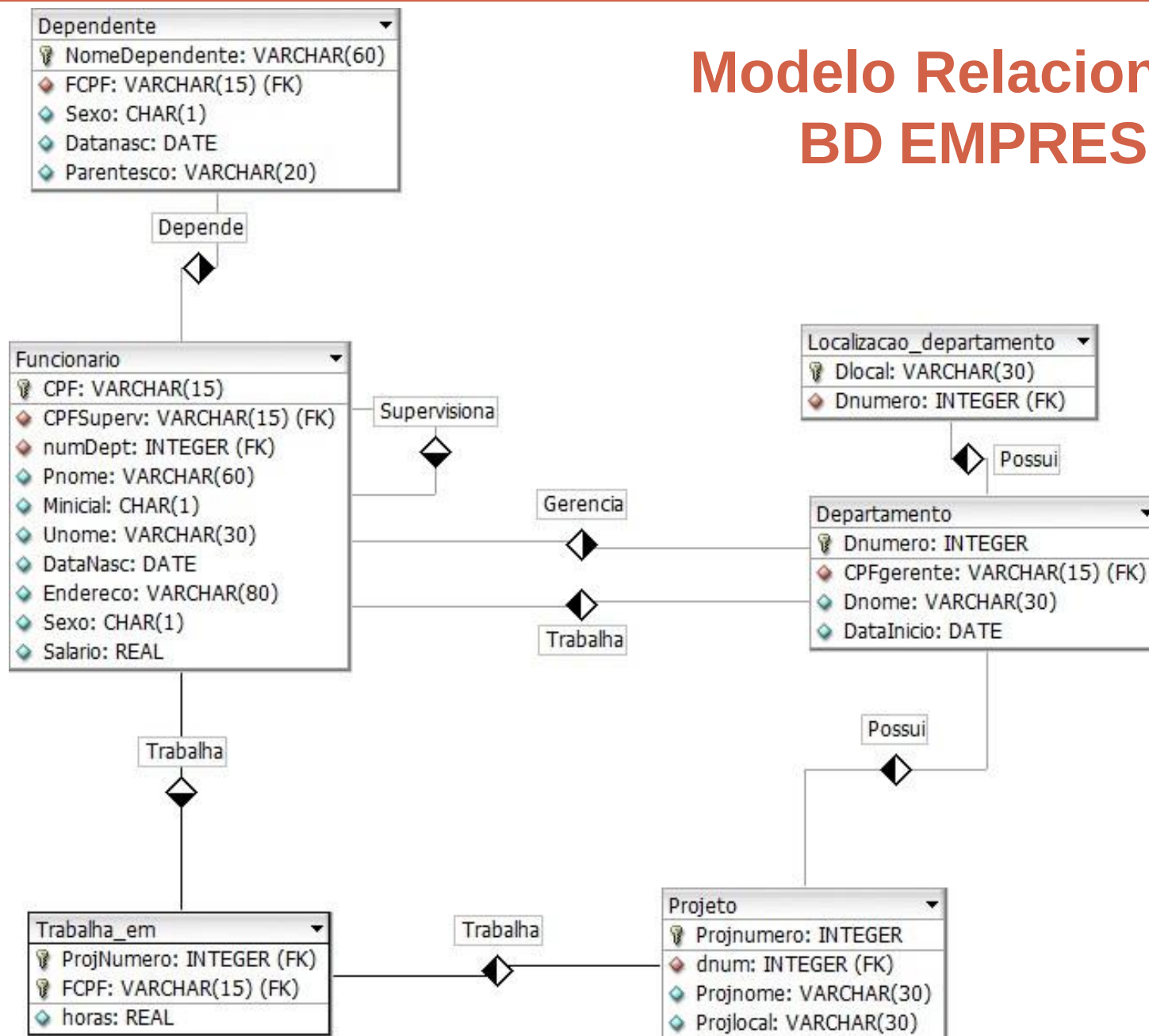
Triggers (gatilhos) - Limitações

- Não é possível criar um Trigger em uma View;
- Não podemos usar em um Trigger os seguintes comandos SQL:
 - ALTER DATABASE; ALTER TRIGGER; ALTER PROCEDURE; ALTER TABLE; ALTER VIEW; CREATE DATABASE; CREATE INDEX; CREATE PROCEDURE; CREATE SCHEMA; CREATE TABLE; DROP DATABASE; DROP TABLE; DROP PROCEDURE; DROP TRIGGER; DROP INDEX; GRANT; REVOKE.

Exemplo

- Inserir um registro na tabela de log **FuncionarioLog (FunCPF, Comando, Usuario, Data)** toda vez que um novo funcionário for inserido na tabela funcionário.
- Vamos ao modelo...

Modelo Relacional do BD EMPRESA



Exemplo – Criar um trigger

```
CREATE TRIGGER tg_Funcionario_Insert  
ON Funcionario FOR AFTER INSERT  
AS  
  
  INSERT INTO FuncionarioLog (FunCPF,  
    Comando, Usuario, Data) VALUES (cpf,  
    'INSERT', CURRENT_USER(),  
    CURRENT_DATE());
```

Exemplo – Alterar um trigger

```
ALTER TRIGGER tg_Funcionario_Insert  
ON Funcionario RENAME TO  
tgFuncionarioInsert;
```

Exemplo – Excluir um trigger

```
DROP TRIGGER tgFuncionarioInsert ON  
Funcionario CASCADE;
```

Habilitar e desabilitar Triggers

- Para desabilitar:

```
ALTER TABLE Funcionario  
DISABLE TRIGGER tgFuncionarioInsert;
```

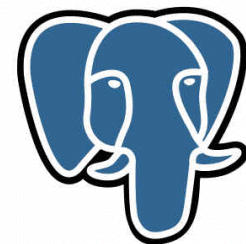
- Para habilitar:

```
ALTER TABLE Funcionario  
ENABLE TRIGGER tgFuncionarioInsert;
```

Observação

- Não é possível criar mais de um *trigger* para o mesmo evento e momento de execução na mesma tabela.
- Por exemplo, não se pode criar dois gatilhos **AFTER INSERT** na mesma tabela.

NA PRÁTICA – SGBDS



Na prática - PostgreSQL

- A diferença das triggers deste SGBD para outros é que no PostgreSQL as triggers são sempre associadas a funções (stored procedures) e, nos demais, criamos o corpo da trigger na própria declaração;
- O PostgreSQL disponibiliza duas variáveis importantes para serem usadas em conjunto com as triggers: NEW e OLD;

Sintaxe (PostgreSQL)

```
CREATE TRIGGER <nome_trigger> {  
BEFORE | AFTER } { evento [ OR ... ]  
} ON <nome_tabela>[ FOR [ EACH ] {  
ROW | STATEMENT } ] EXECUTE PROCEDURE  
nome_da_função();
```

Exemplo – Criar trigger(PostgreSQL)

```
CREATE TRIGGER "tg_funcionario_insert"  
AFTER INSERT ON funcionario  
FOR EACH ROW  
  
EXECUTE PROCEDURE  
insert_funcionario();
```

Alterar trigger (PostgreSQL)

- Sintaxe:

```
ALTER TRIGGER nome ON tabela RENAME  
TO novo_nome
```

- Exemplo:

```
ALTER TRIGGER tg_validaCpf ON  
Funcionario RENAME TO tgValidaCPF;
```

Excluir trigger – PostgreSQL

- Sintaxe:

```
DROP TRIGGER nome ON tabela [CASCADE  
| RESTRICT]
```

- Exemplo

```
DROP TRIGGER tg_validaCpf ON  
Funcionario CASCADE;
```

Habilitar/Desabilitar trigger - PostgreSQL

- Desabilitar

```
ALTER TABLE nome_tabela DISABLE  
TRIGGER nome_trigger
```

- Habilitar

```
ALTER TABLE nome_tabela ENABLE  
TRIGGER nome_trigger
```

Usando o Catálogo de Dados

- Obtendo informações sobre as triggers do banco de dados

SELECT * FROM

INFORMATION_SCHEMA.TRIGGERS;

Nome	Descrição
trigger_catalog	Nome do banco de dados que contém a trigger.
trigger_schema	Nome do esquema que contém a trigger.
trigger_name	Nome da trigger.
event_manipulation	O evento que dispara a trigger.
event_object_catalog	Nome do banco de dados que contém a tabela onde a trigger está definida.
event_object_schema	Nome do esquema que contém a tabela onde a trigger está definida.
event_object_table	Nome da tabela onde a trigger está definida.
action_statement	Declaração executada pela trigger.
action_orientation	Identifica se a trigger dispara uma vez para cada linha processada ou uma vez para cada declaração (ROW ou STATEMENT)
condition_timing	Momento em que a trigger dispara (BEFORE ou AFTER)



Rules – PostgreSQL

- O PostgreSQL tem implementado um comando chamado Rules (regras), que são semelhantes aos triggers (SQL);
- Algumas diferenças:
 - Com rules podemos substituir (INSTEAD) ou desabilitar (NOTHING) comandos de maneira transparente ao usuário e sem abortar a transação;
 - As rules são executadas antes do início da execução de um comando;

Rules – PostgreSQL

- Sintaxe:

```
CREATE [OR REPLACE] RULE <nome_rule>  
AS ON <evento> TO <nome_tabela>  
[WHERE condição] DO [INSTEAD] ação
```

- Onde a ação pode ser:
 - NOTHING
 - Blocos de sqls

Rules – PostgreSQL

- Exemplo
 - Elabore uma rule que armazene as informações de deleção de todos os clientes.

```
CREATE RULE rl_customers_delete AS  
ON DELETE TO customers DO  
INSERT INTO log_customers (name,  
acao, usuario) VALUES (old.name,  
'DELETE', CURRENT_USER);
```

Rules – PostgreSQL

```
CREATE TABLE log_customers (  
    id serial PRIMARY KEY  
    name varchar(50),  
    usuário varchar(50),  
    acao varchar(20),  
    data TIMESTAMP DEFAULT  
        CURRENT_TIMESTAMP  
) ;
```

Gerenciamento de Rules – PostgreSQL

- Excluir

```
DROP RULE rl_customer_delete ON  
customers CASCADE;
```

- Comando para verificar quais rules fazem parte de uma tabela

```
SELECT * FROM pg_rules WHERE  
tablename='customers';
```

Na prática – MySQL



- O suporte a triggers foi incluído na versão 5.0.2 do MySQL;
- Suporta o uso dos registros NEW e OLD;

Na prática - MySQL

- Sintaxe de criação

```
CREATE TRIGGER <nome_trigger> [BEFORE  
  | AFTER] evento ON <nome_tabela>  
FOR EACH ROW  
BEGIN  
/*corpo do código*/  
END
```

Exemplo – Criar trigger(MySQL)

```
DELIMITER $$
```

```
CREATE TRIGGER tg_funcionario_insert  
AFTER INSERT ON Funcionario  
FOR EACH ROW  
BEGIN  
    INSERT INTO FuncionarioLog(FunCPF,  
    Comando, Usuario, Data) VALUES (cpf,  
    'INSERT', CURRENT_USER(),  
    CURRENT_DATE());  
END$$
```

```
DELIMITER ;
```

Exemplo – Excluir trigger (MySQL)

- Sintaxe:

```
DROP TRIGGER nome_trigger;
```

- Exemplo:

```
DROP TRIGGER tg_funcionario_insert;
```


Observações – MySQL

- O MySQL não suporta o comando `ALTER TRIGGER;`
- Comando para visualizar triggers no banco de dados;
SHOW `triggers;`
- Habilitar/Desabilitar triggers
ALTER TABLE `<nome_tabela>`
DISABLE/ENABLE TRIGGER ALL

Na prática – SQL Server



- No SQL Server existem 2 tipos de triggers : **AFTER** (executada depois da gravação dos dados no banco) e **INSTEAD OF** (a trigger será disparada antes do registro ser modificado);
- Existem duas tabelas de ambiente na execução de uma trigger no SQL Server:
 - **INSERTED** (registros incluído ou alterados – novos valores);
 - **DELETED** (registros excluídos ou alterados – valores antigos);
 - Estas tabelas “virtuais” possuem exatamente a mesma estrutura da tabela em que a trigger foi criada.

Sintaxe – Criar trigger (SQL Server)

```
CREATE TRIGGER <nome_trigger>  
ON <nome_tabela>  
{ FOR / AFTER | INSTEAD OF }  
{ [INSERT] [,] [DELETE] [,] [UPDATE]  
}  
AS  
BEGIN  
    comandos_sql [ ...n ]  
END
```

Exemplo – Criar Trigger (SQL Server)

```
CREATE TRIGGER tg_funcionario_insert
ON funcionario FOR INSERT AS
BEGIN
    DECLARE @fcpf INT
    SELECT @fcpf = cpf FROM INSERTED;
    INSERT INTO FuncionarioLog (FunCPF,
Comando, Usuario, Data) VALUES
(@fcpf, 'INSERT', CURRENT_USER(),
CURRENT_DATE());
END
```

Alterar trigger – SQL Server

```
ALTER TRIGGER <nome_trigger>  
ON <nome_tabela>  
{ FOR / AFTER | INSTEAD OF }  
{ [INSERT] [,] [DELETE] [,] [UPDATE]  
}  
AS  
BEGIN  
    comandos_sql [ ...n ]  
END
```

Excluir trigger – SQL Server

- **Sintaxe:**

```
DROP TRIGGER <nome_trigger>
```

- **Exemplo:**

```
DROP TRIGGER tg_funcionario_insert;
```

Habilitar/Desabilitar triggers – SQLServer

- **Habilitar**

```
ALTER TABLE <nome_tabela> ENABLE  
TRIGGER <nome_trigger>
```

- **Desabilitar**

```
ALTER TABLE <nome_tabela> DISABLE  
TRIGGER <nome_trigger>
```

Observações

- A trigger no SQL Server é disparada por evento e não por número de linhas afetadas;
- Para tratar todos os registros da tabela deve-se utilizar o conceito de cursor (que será visto na próxima aula);



Dúvidas??????????



Referências

- ELMASRI, R; NAVATHE, S. B. **Sistemas de Bancos de Dados**. São Paulo. Addison Wesley, 4ed, 2005.
- RAMAKRISHNAN, R. **Sistema de Banco de Dados**, McGraw-Hill, 1999

